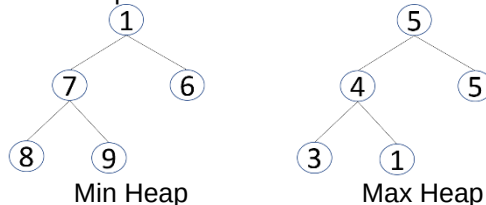


Heaps and Priority Queues

Heaps

- A **heap** is a complete binary tree where the value of each of each parent node is either higher or lower than the value of its child nodes.
- These are the two (2) types of heap:
 - Min Heap** – The value of each parent node is less than or equal to the values of its child nodes.
 - Max Heap** – The value of each parent node is greater than or equal to the values of its child nodes.



- Heaps can be implemented in Java using **ArrayList** from the **java.util** package.
Example:
`ArrayList<Integer> heap = new ArrayList<>();`
- To create a heap with initial values, use the **addAll()** method of the **Collections** class (also from the **java.util** package).
Example (based on the given min heap above):
`ArrayList<Integer> minHeap = new ArrayList<>();`
`Collections.addAll(minHeap, 1, 7, 6, 8, 9);`
Explanation: The root node is always at index 0. Its child nodes are always at index 1 and index 2. Indices 3, 4, 5, and 6 can store the child nodes of those two (2) nodes and so on.
- To determine the root node, use the **get()** method.
Example: `System.out.print(minHeap.get(0));`

Priority Queues

- A **priority queue** is a special type of queue where elements are processed based on their order (natural or custom).
- Priority queues can be implemented in Java using the **PriorityQueue** class from the **java.util** package.
Example:
`PriorityQueue<Integer> printer = new PriorityQueue<>();`
- To enqueue, use **add()** or **offer()**. To dequeue, use **poll()**

or **remove()**. The dequeue operation starts with the minimum element.

- Sample code to dequeue in Java based on natural order:
`PriorityQueue<Integer> printer = new PriorityQueue<>();`
`printer.add(9);`
`printer.add(7);`
`printer.add(1);`
`printer.add(3);`
`while (!printer.isEmpty()) {`
`System.out.print(printer.remove() + " ");`
`}`
Output: 1 3 7 9
- Sample code to dequeue in Python based on natural order:
`from queue import PriorityQueue`
`prio = PriorityQueue()`
`prio.put("dog")`
`prio.put("d")`
`prio.put("dogs")`
`while not prio.empty():`
`next_item = prio.get()`
`print(next_item)`
- A **comparator** is used to create a specific ordering for a collection of objects.
- To create a comparator in Java, use the **Comparator** interface and its methods such as **comparing()**, **comparingInt()**, and **compare()**.
- Sample code to dequeue in Java based on a custom order:
`Comparator<String> comp = Comparator.comparing(String::length);`
`//:: indicates a method reference, ContainingType::methodName`
`PriorityQueue<String> prio = new PriorityQueue<>(comp);`
`prio.add("cat");`
`prio.add("c");`
`prio.add("cats");`
`while (!prio.isEmpty()) {`
`System.out.println(prio.remove());`
`}`

References:

- Koffman, E. & Wolfgang, P. (2016). *Data structures: Abstraction and design using Java*. Hoboken: John Wiley & Sons, Inc.
- Oracle Docs. (n.d.). *Citing sources*. Retrieved from <https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html>
- Python Software Foundation. (n.d.). *The Python tutorial*. Retrieved from <https://docs.python.org/3/tutorial/index.html>